

迭代评估报告

评估日期：2026. 07. 23

组号	07	项目名称	GitHub 仓库问答与 Issue 分析系统
迭代名称	技术原型迭代	实际起止日期	2026 年 7 月 14 日 - 2026 年 7 月 24 日
任务达成情况：（完成的任务和实现的需求等） 本轮迭代超额完成基础底座与核心高难度模块开发，核心业务链路已跑通，具体完成情况如下： 1. 文档与架构设计全部落地 完成技术选型与架构定稿：确认 Spring Boot 后端技术栈、RAG+GraphRAG+AST 增强的核心算法方案、四层分层架构，完成核心技术可行性验证。 输出全套架构设计资产：完成用例、逻辑、部署、数据、算法、可维护性六类 UML 视图，完成数据库物理模型设计，架构文档迭代至 V1.1 版本。 2. 后端核心底座 100% 搭建完成 完成后端工程完整实现：基于 Spring Boot + JPA + OpenAPI 搭建后端骨架，支持 H2 开发环境与 MySQL 生产环境双数据源，配套数据库迁移脚本，兼容历史数据。 完成统一用户体系与鉴权：落地 JWT 认证机制，实现 owner_login 维度的数据隔离、按用户仓库的权限隔离，支持启动自动创建默认管理员账号，完整支撑 UC0 用户登录用例。 完成基础工程能力补全：全局异常处理、跨域 CORS 支持、环境变量自动加载、操作日志体系、构建进度反馈等公共能力全部落地，支撑全模块复用。 3. 核心高难度模块（UC2/UC3）取得突破性进展 知识库构建链路全量跑通：完成代码文件解析、代码摘要生成、向量化语义检索全流程，落地 AST 语法树解析、GraphRAG 增强检索能力，大幅提升代码场景的检索准确率，完整覆盖 UC2 仓库代码知识库自动构建核心需求。 多维度检索能力成型：向量语义检索 + 语法结构检索双引擎落地，为 UC3 AI 代码智能问答提供了完整的检索底座。			

- FAQ 自动生成能力落地：基于知识库内容自动聚类生成 FAQ 条目，直接支撑 UC6 仓库长期记忆与知识库维护用例。

4. 多用户与部署能力完成改造

- 完成多用户云部署适配：实现用户级数据隔离、仓库级权限隔离，新增并发锁优化避免重复构建，支持前端资源打包进 JAR 单体部署，具备上云部署基础。
- 工程化能力完善：修复开发环境 Vite 监听、跨域配置、环境变量加载等工程问题，开发体验与部署便捷性显著提升。

5. 待完善项（计划内顺延优化）

- 大体积仓库构建耗时较长，极端场景存在卡死风险，当前为同步构建模式，并行优化待落地；
- 邮件通知、告警类功能暂未实现；
- 云服务器环境的全量部署与压测尚未完成验证；
- 多用户、多仓库的并行构建能力待进一步优化。

质量情况：（执行了哪些评审和测试？评审和测试的结果如何？）

1. 评审执行情况

- **代码 PR 评审：**本轮迭代所有功能均通过功能分支提交，发起 Pull Request 并申请交叉评审，提交记录完整可追溯；针对知识库构建、多用户改造等核心模块进行了重点代码审查。
- **架构与方案评审：**架构文档经过全员评审定稿；大仓库性能、并行方案等技术点经过专项讨论，形成了明确的后续优化路径。
- **评审结论：**架构设计合理，代码实现符合分层规范，核心模块思路清晰，高风险点已识别并形成预案，评审通过。

2. 测试执行情况

- **模块级自测全覆盖：**对用户认证、知识库构建、数据隔离、FAQ 生成、数据库迁移

等核心模块逐一开展功能自测，验证业务逻辑正确性与数据一致性。

- **缺陷修复闭环：**本轮共发现并修复多项核心缺陷，包括：知识库构建文件 EOF 异常、H2 schema 不同步问题、跨域配置失效、环境变量未自动加载、默认账号未自动创建等，所有已发现缺陷均完成修复与回归验证。
- **测试结论：**核心功能链路运行稳定，基础场景验证通过；大仓库性能、极端并发场景属于已知优化项，纳入后续迭代，不影响本轮技术原型的核心可用性。

问题、变更和返工：（遇到的问题、发生的变更、是否需要返工等）

1. 已解决的问题与变更

- **数据源适配变更：**初期仅支持 H2 内存数据库，后补充 MySQL 生产环境支持，并新增 ALTER TABLE 迁移脚本兼容旧数据。属于功能扩展，未造成返工。
- **用户体系重构：**从分散用户管理调整为统一 app_users 用户中心，同步完成数据隔离逻辑改造。属于设计优化，在模块开发早期完成调整，未造成大规模返工。
- **工程问题修复：**跨域、环境变量、开发服务器监听等工程问题，均为局部配置修复，影响范围小，已快速闭环。
- **知识库构建异常修复：**大文件读取 EOF 报错，通过优化文件读取逻辑解决，未影响核心架构。

2. 剩余待解决问题

1. **大仓库构建性能问题：**大体积仓库同步与构建耗时长，同步模式下存在阻塞风险，需改造为异步任务 + 并行分片构建。
2. **功能缺口：**邮件通知、告警推送功能暂未实现。
3. **部署验证缺口：**仅在本地环境完成验证，云服务器环境的全量部署、性能压测尚未执行。
4. **并发能力待强化：**当前已通过并发锁避免重复构建，但多用户、多仓库的并行构建能力未完全落地，高并发场景性能待优化。

3. 返工情况说明

本轮无架构级、模块级的大规模返工；所有变更均为迭代中的优化与扩展，在计划可控范围内，未对整体进度造成严重影响。剩余问题均为已知优化项，纳入下一迭代排期，无需本轮

返工。

经验和教训：

1. 经验总结

- **「核心链路优先跑通」的策略高效可行：**优先攻坚知识库构建、用户鉴权等高难度核心模块，尽早暴露技术风险，比按顺序线性开发更能把控整体风险。
- **数据隔离设计前置非常必要：**在开发早期就落地多用户数据隔离与权限体系，避免了后续改造带来的大量返工，为后续上云与多用户场景打下了良好基础。
- **工程化能力前置能显著提效：**提前解决跨域、环境变量、数据库迁移等工程问题，让后续业务模块开发更顺畅，减少了大量环境调试成本。
- **AST+GraphRAG 增强检索方向正确：**在纯向量检索基础上补充语法结构与图关系检索，能明显提升代码场景的检索质量，符合项目技术预期。

2. 教训与改进方向

- **异步并行架构需更早规划：**大仓库构建耗时问题暴露了同步模式的瓶颈，后续迭代需优先引入任务队列与异步流水线，从架构层面解决性能与卡死风险。
- **部署验证需提前介入：**不能仅满足于本地运行，后续应尽早启动云环境部署验证，提前暴露环境差异带来的问题。
- **非核心功能做好优先级排序：**邮件通知等辅助功能优先级后置，优先保障核心业务链路稳定，符合迭代原则；后续按需逐步补充即可。
- **并发与性能设计需前置考虑：**多用户、多仓库场景的并行能力，应在架构设计阶段做更充分的预案，避免后期改造成本过高。

